Paxos Made Simple

Introduction to Paxos

Paxos is a **consensus algorithm** that allows a set of processes to agree on a single value among the proposed values.
Paxos assumes **asynchronous systems**, in which processes can **crash**.

Properties

Validity: If a process decides a value **v**, then **v** was proposed by some process. The protocol does not invent values.
Integrity: No process can decide twice. Once a decision is made, it is final.
Agreement: No pair of processes can decide different values. All processes decide the same global value.
Termination: Every correct process eventually decides a value.
Paxos is designed to guarantee **safety**, meaning that, even in any chaotic situation, the system must continue to make correct decisions. Therefore, it is it guarantees **validity**, **integrity**, and **agreement**.

Liveness, meaning **termination**, is not always guaranteed. It is guaranteed only when the network is **eventually synchronous**, after the stabilization time **T**, when the network becomes synchronous.

Paxos Roles

Proposers: Processes that propose a value for consensus. There can be multiple proposers at the same time.
Acceptors: Voting processes that accept a final value among the proposed ones. They receive **prepare** requests from proposers, make **promises**, accept values, and reject "old" proposals.
Learners: Learners learn the final chosen value.
It is assumed that there is a correct majority only in the set of **acceptors**. Proposers and learners may also crash.
Chosen: A learner must not learn that a value has been decided unless it has really been chosen. A value is **chosen** when a majority of acceptors has accepted it. The decision is irreversible. A value is **learned** when the learners have learned it.

Complete Paxos Sequence

Phase 1a — Prepare: The proposer chooses a number **n** and sends **PREPARE(n)** to a majority of acceptors.
Phase 1b — Promise: When an acceptor receives a **PREPARE(n)** request, if **n** is greater than the number of every **PREPARE** request to which it has already replied, the acceptor replies with a **PROMISE**. It promises not to accept any proposal with an identifier lower than **n**.
Then, if it exists, the acceptor returns the value **v'** related to the accepted proposal with the highest proposal number **ACK(n, n', v').**
Otherwise, if it has never accepted any proposal, it returns **ACK(n, ⊥, ⊥).**
The proposal **n'** is not necessarily **chosen**. It may have been accepted only by that acceptor.
If **n** is old, meaning **n < n'**, the acceptor rejects the proposal by sending **NACK(n')**.

Phase 2a — Accept: The proposer collects the replies from the acceptors. Among the replies, it sets its value **v** to the value **v_prev** contained in the reply with the highest proposal number, if such a value exists. Otherwise, it chooses its own value **v**.
If a majority of acceptors has replied with a **PROMISE**, the proposer sends the request **ACCEPT(n, v)**.
Example: The proposer receives the following accepted proposals from the acceptors: **a1 → ⟨3, 0⟩**, **a2 → ⟨7, 1⟩**, **a3 → ⊥**. It must choose **v = 1**, because **n = 7** is the highest proposal number among the accepted proposals.
Phase 2b — Accepted: When an acceptor receives an **ACCEPT(n, v)** request, it accepts the proposal unless it has already replied to a **PREPARE** request with a number greater than **n**.

Decision and Chosen Value
A value is **chosen** when a majority of acceptors accepts **⟨n, v⟩**. When an acceptor accepts a proposal, it sends **ACCEPTED(n, v)** to the learners. When a learner receives **ACCEPTED(n, v)** from a majority of acceptors, it can conclude that the value **v** has been chosen, and therefore learns it as the decided value.
Number of Messages
In the **best case**, a single Paxos instance uses **4n messages**, where **n** is the number of acceptors. There is one **PREPARE** message sent from the proposer to all acceptors, one **Promise ACK** sent back from the acceptors to the proposer, one **ACCEPT** message sent from the proposer to all acceptors, and one **ACCEPTED** message sent back from the acceptors, at least from a majority, to the proposer.
In the **worst case,** the number of messages can be potentially infinite, because proposers can continuously overtake each other.

Worst Case Example: A starts **Prepare(2)** and receives the **Promises**. Before it can complete **Phase 2**, B arrives and starts **Prepare(4)**. The acceptors now promise proposal number **4**. Therefore, A's following **AcceptRequest(2, 8)** is rejected.
A then tries again with a higher number: A starts **Prepare(6)**, but before it finishes, B tries again: B starts **Prepare(8)**. And so on.

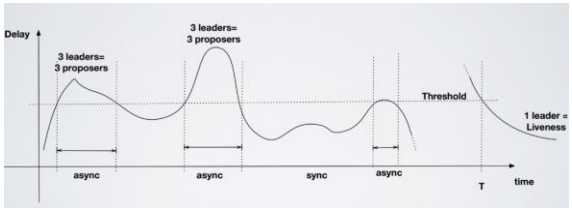
Proof of Agreement for Safety
We want to prove **agreement**, meaning that at most one value can be **chosen**, and therefore **two** different **values cannot** be **chosen**.
Let a **ballot** be a pair **⟨proposalNumber, value⟩**. For a ballot to be **chosen**, it must be **accepted** by a majority **quorum**, and any two quorums intersect.
If a ballot **b = ⟨proposalNumber, v⟩** is proposed for acceptance, either no value was chosen before **b**, or all previously chosen ballots already had value **v**. We proceed by **contradiction**.

Suppose that there is a ballot **b** that chooses value **v**, written **(b, v)**, and a ballot **b'** that chooses value **v'**, written **(b', v')**, with **v ≠ v'** and **b < b'**.
"**b' chooses v'**" means that a **quorum accepted** (b', v'). Therefore, the proposer of **b'** queried the quorum, collected the latest accepted proposals, and **selected** the **value** connected to the **proposal** with the **highest** ID, or **proposed** a **value** if **no previous** proposal had been accepted.

If no previous proposal had been accepted, **b'** chooses **v'** by proposing it. However, this is a **contradiction**, because we assume that proposal **b**, containing **v**, had **already** been **accepted** before.
If the value connected to the **previously accepted proposal** with the **highest** ID is **selected**, there is also a contradiction. We assume that **b'** chooses **v'**, which is the dominant value among the previous ballots. However, the **dominant value** is **v**, because a value **v** had **already** been **chosen** before. Therefore, value **v** is **selected** for ballot **b'**.
This is a **contradiction**, because we assumed that **b'** chooses **v' ≠ v**. Therefore, the **property** is **proved**.

Liveness Problem
Liveness, meaning whether the system eventually decides something, cannot be proved. According to **FLP**, it is not possible to guarantee both **safety** and **liveness** in a consensus algorithm when the system is asynchronous. Paxos guarantees **safety**, but it does not always guarantee **liveness**.
As explained in the previous example, proposers can continuously prevent each other from completing the protocol, and no value is chosen. However, it is guaranteed that the system never enters an inconsistent state.

Liveness Solution
During the asynchronous phase, there may be multiple processes that consider themselves leader proposers. After the stabilization time, however, the system identifies a single stable leader proposer, which is able to complete the protocol without being continuously overtaken by other proposers.
Under these conditions, Paxos guarantees **liveness**, because no other proposer interrupts the leader proposer.



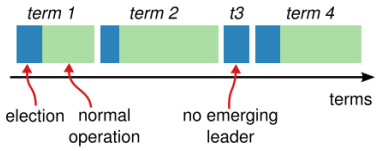
Raft: Consensus Through a Leader and a Replicated Log

Premises

There is a system with multiple server nodes, in which all nodes must execute the same commands in the same order. Some nodes may fail, and the network may lose messages or deliver them after an indefinite delay.
Each node stores a **log**, which is an ordered sequence of commands. The **Raft algorithm** is used to reach consensus on the replicated log among the nodes of the system.
When the network reaches consensus on a command in the log, the command is applied locally by the node. Raft assigns the management of consensus to a single **leader server**, which controls the log.

Node Roles

Follower: A passive node that receives **RPC calls** from the leader. It does not propose new entries. If a client contacts a follower, the follower redirects it to the leader.
Candidate: A node that tries to be elected as leader when the current leader crashes.
Leader: The node that manages client requests and log replication. The leader receives commands from clients, adds them to its own log, replicates them to the followers, and informs the followers when the commands are confirmed and can be executed.
If the leader fails, a new leader is elected. There is only one leader, while the other servers are followers.



Term Number

Each node maintains a logical clock called a **term**, which is incremented every time a new election starts. There can be at most one valid leader for each **term** (epoch).
When two servers communicate, they include their current term in the messages. If a node receives a term greater than its own, it updates its term to the received one. If it receives a request with a lower term, it rejects it. If a leader or a candidate discovers that it belongs to an outdated term, it immediately loses its role and becomes a follower again.
This prevents an old isolated leader from continuing to be considered valid after the cluster has elected a new leader.

RPCs Used

Two types of **RPCs** are used. **RequestVote()** is used by candidates during elections to request votes from the other nodes.
AppendEntries() is used by the leader to replicate the log and send heartbeats to the other nodes.
The RPCs are sent in parallel to all nodes and are retried when no response is received.

Leader Election

Failure Detection: The leader periodically sends **heartbeats** to the followers, which are empty **AppendEntries RPCs** used to communicate that the leader is still operating.

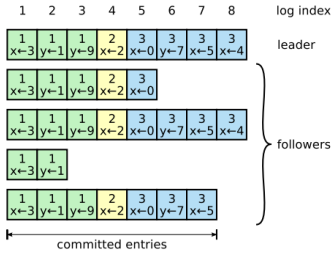
Each follower has an **election timeout**. If the timeout expires without the follower receiving a heartbeat from the leader, the follower assumes that the leader has crashed and starts a new election. The leader may only be delayed, but it is still removed from its role.

Election: After the timeout expires, to start an election, the follower increments its own **term**, changes to the **candidate** state, votes for itself, and sends a **RequestVote RPC** to all the other nodes. During a term, each server can vote at most once, for the first candidate from which it receives a **RequestVote**. A candidate becomes leader if it obtains votes from a majority of the nodes.
Election Victory: If a candidate obtains the majority of votes, it becomes leader and immediately sends a heartbeat to the other servers, communicating its authority.

Discovery of a Valid Leader: While waiting for votes, the candidate may receive an **AppendEntries** message from another server claiming to be the leader. If the term indicated in the request is at least equal to its own term, the candidate recognizes the leader and becomes a follower again. This is the case in which the candidate does not receive messages because of network delays, while another node has been elected leader in the meantime.
If the term is lower, the candidate considers the message obsolete and rejects it. This is the case in which an old leader, previously blocked by network delays, becomes active again.

Competing Candidates: In the same term, for example, if two nodes reach their timeout and become candidates at the same time, two leaders cannot be elected. Because majorities intersect, a follower can join one majority and vote for one candidate, or join the other majority and vote for the other candidate.
If the votes are split equally and no candidate obtains a majority, when the next timeout expires, the candidates increment the term and start a new election.
To avoid continuous elections, Raft uses random timeouts for each follower, between **150 and 300 milliseconds**, so that only one follower reaches the timeout first and becomes a candidate.

Log Replication
When a command arrives from a client, the leader creates a new **entry** in its own log, assigns the current **term** and the corresponding log **index** to the entry, and sends the **AppendEntries() RPC** to the followers. The leader then waits until the entry is stored by a majority of the nodes before marking it as **committed**. Once the entry is committed, both the leader and the followers execute the command.
The leader can reply to the client after the commit, without waiting for all nodes to be updated.

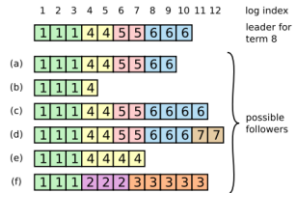


Entry State Distinction
To distinguish, when receiving **AppendEntries**, which entries are committed and which are new entries that only need to be stored, the leader includes a **commitIndex**. This represents the highest-indexed committed entry.
Entries after this index are new entries that only need to be stored. For entries before this index, the commands contained in them can be applied.

Retry
If a follower is slow or faulty, the leader continues retrying **AppendEntries** until the follower recovers the missing entries.

Log Matching Property
The goal is to guarantee that, if two entries in different logs have the same index and the same term, then they contain the same command, and that, if two logs contain an entry with the same index and the same term, then they also match in all previous positions.
The first property follows from the fact that, during a term, the leader creates at most one entry for each index, and the entry contains only one command. All followers must apply only the entries committed by the leader.
The second property is guaranteed by the **AppendEntries consistency check**.

AppendEntries Consistency Check
When the leader sends new entries, it also includes the index of the entry immediately preceding the new ones, **prevLogIndex**, and the term of that entry, **prevLogTerm**.
The follower accepts the new entries only if, in its own log, it has an entry at position **prevLogIndex** with the same term **prevLogTerm**. If the entry is missing or belongs to a different term, the follower rejects the request.
If the follower accepts, the leader knows that the follower's log matches its own up to the entries that have just been replicated.
Example: The leader wants to send entry **8** and communicates **prevLogIndex = 7, prevLogTerm = 3.** The follower checks its own entry **7**. If entry **7** exists and belongs to term **3**, it accepts the continuation; otherwise, it rejects it.



Inconsistent Logs
When a leader fails, it may not yet have replicated all the entries in its log to all the nodes.
After several leader changes, in a new term, a follower may be missing some entries that are present in the leader's log, or it may contain some entries that are not present in the leader's log.
To **solve** this **problem**, the leader forces the followers to make their logs equal to its own, by deleting conflicting entries and learning the correct ones.

Repair

For each follower, the leader stores an index called **nextIndex**, which indicates the next entry to send to that follower. When a node becomes leader, it initializes **nextIndex** to the position immediately after the last entry in its own log.

To understand where each follower's log diverges from its own log, the leader sends an **AppendEntries** request starting from **nextIndex**. If the follower rejects it, this means that the two logs do not match at the previous position. The leader decreases **nextIndex** and retries the request, continuing in this way until it finds the last position on which the two logs agree. From that point, the follower deletes the conflicting entries and adds the entries received from the leader.

Example: Suppose the logs are: **Leader:** A B C D E, **Follower:** A B X Y. The consistency check fails in the final positions. The leader moves backward until it discovers that both logs agree on **A B**. The follower deletes **X Y** and receives **C D E**, obtaining: **Follower:** A B C D E

Inconsistency Caused by an Outdated Leader

Consider the case in which a leader replicates some entries to a majority of the nodes, while one follower remains disconnected and does not receive those entries. The leader fails, and the outdated follower is elected as the new leader. The new leader creates other entries in the same positions, and the previous committed and applied entries are overwritten. In this way, different nodes could apply different commands at the same index, violating **safety**.

To prevent this, during the election, a candidate can receive a vote only if its log is at least as up to date as the log of the voting node. Otherwise, it cannot receive the vote.

Between two logs, the more up-to-date log is considered to be the one with the higher final term or, if the final terms are equal, the one with more entries, and therefore with the higher final index. The pairs **(lastLogTerm, lastLogIndex)** of the two logs are therefore compared.

Example: Log A: last term = 5, last index = 8. **Log B:** last term = 4, last index = 20. **Log A** is considered more up to date.

Instead, if: **Log A:** last term = 5, last index = 8, **Log B:** last term = 5, last index = 11, then **Log B** is considered more up to date.

This criterion is used because, if an entry is **committed**, it is present on a majority of the nodes. The candidate must receive votes from another majority to be elected. The two majorities have at least one node in common because of quorum intersection. Therefore, that node will not vote for a candidate whose log is too outdated, and such a candidate will never obtain a majority.

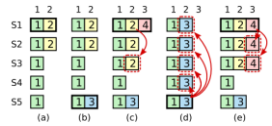
Committing Entries from Previous Terms

A leader can directly mark as **committed** only an entry that belongs to its current term.

An entry from a previous term, created by a previous leader, may be stored on a majority of the nodes but still not be committed, because the leader crashed or because some nodes had not received the entry. The new leader cannot commit it only on the basis of the majority rule, because the entry does not belong to its current term.

The new leader must replicate at least one entry from its current term. When this entry reaches a majority and becomes **committed**, all the previous entries in the leader's log are also recovered and indirectly committed.

Example: Suppose we have five servers: **A, B, C, D, E**. In **term 2**, leader **A** adds entry **4, term 2, command X** to the log. The entry is replicated to **A, B, and C**, so it is present on a majority of three servers. However, **A** fails before safely communicating the **commit**. We have:
A: [X, term 2], **B: [X, term 2]**, **C: [X, term 2]**, **D: [ ]**, **E: [ ].**
In **term 3**, **B** is elected leader. Even if **X** is present on the majority, **B** cannot advance the **commitIndex** based only on **X**, because **X** belongs to **term 2**.
B receives a new command **Y** and creates **entry 5, term 3, command Y**. **B** replicates **Y** to **C** and **D**: **B: [X, term 2, Y, term 3]**, **C: [X, term 2, Y, term 3]**, **D: [X, term 2, Y, term 3].**
Now **Y**, which belongs to **term 3**, is present on the majority. Therefore, **B** can declare **Y committed**. Since **X** appears before **Y** in the leader's log, **X** is also considered committed.
Therefore, **X** is committed indirectly, while **Y** is committed directly.



Raft Properties
Election Safety: At most one leader can be elected in a given term.
Leader Append-Only: A leader never overwrites or deletes entries in its own log; it can only add new entries.
Log Matching: If two logs contain an entry with the same index and the same term, then the two logs are identical in all entries up to and including that index.
Leader Completeness: If a log entry is committed in a given term, it will be present in the logs of all leaders elected in later terms.
State Machine Safety: If a server has applied a log entry at a given index to its state machine, no other server will ever apply a different entry at the same index.

Proof of the Leader Completeness Property
We want to prove that, if an entry is **committed** in a certain term, it will be present in the log of every leader elected in later terms.
By contradiction, suppose that in term **T**, leader **leader_T** replicates an entry from its own term to a majority of the servers, making it committed, but there is a later term **U > T** whose leader **leader_U** does not contain this entry.
Let **U** be the first term in which this happens. Therefore, all leaders elected between **T** and **U** contained the entry.
The entry must already have been missing from the log of **leader_U** at the time of its election, because a leader never deletes or overwrites entries in its own log.
Leader **leader_T** had replicated the entry to a majority of the servers, while **leader_U** was elected by receiving votes from another majority. Since two majorities always intersect, there is at least one server, called the **voter**, that had received the entry from **leader_T** and later voted for **leader_U**.
The voter must have received the entry before voting for **leader_U**. In fact, if it had already participated in term **U**, it would have rejected the messages from **leader_T**, because they came from the previous term **T**.

Moreover, when it voted, the voter still contained the entry. A follower deletes entries only when they conflict with the entries of the leader, but all previous leaders contained that entry. Therefore, no leader before **leader_U** could have deleted the entry from the voter's log.

To grant its vote to **leader_U**, the voter must have checked that **leader_U's log** was at least as up to date as its own. At this point, there are **two** possible **cases**.

1. The two logs end with entries from the same term. In this case, **leader_U's log** must be at least as long as the voter's log. Therefore, it must also contain the committed entry from term **T**. This **contradicts** the hypothesis that **leader_U** does not contain it.
2. The last entry of **leader_U** belongs to a term greater than the term of the voter's last entry. That term must also be greater than **T**, because the voter's log contains at least the entry from term **T**. The last entry of **leader_U** was created by an intermediate leader that, by the choice of **U**, contained the committed entry. By the **Log Matching Property**, if **leader_U** contains that later entry, it must also contain all the entries that precede it, including the entry committed in term **T**. This also **contradicts** the initial hypothesis.

In both cases, we obtain a **contradiction**. Therefore, the property is **proved**.

Proof of the State Machine Safety Property

We want to prove that, if a server applies an entry to its **state machine** at a certain index, no other server will ever apply a different command at the same index.

Consider an index **i** and the first term **T** in which a server applies an entry **x** at position **i**.

When the node applies **x**, two conditions hold. First, **x** is **committed**, because Raft applies only committed entries. Second, the server's log, at least up to index **i**, matches the log of the leader in term **T**.

In fact, a follower learns from the leader which entries are committed through **AppendEntries**, and it accepts the message only if its own log is consistent with the leader's log.

Now consider another node that applies the entry at index **i**.

If it applies it in the same term **T**, it receives the commit information from the same leader. Therefore, by the **Log Matching Property**, it has the same entry **x** at position **i**.

If it applies it in a later term, by the **Leader Completeness Property**, all later leaders contain the committed entry **x** at index **i**. The log of the server applying the entry must also match the log of its own leader up to that index. Therefore, it will apply **x** again.

Thus, **in no case** can a server apply a different entry **y** at the same index **i**. Therefore, the **State Machine Safety Property** is **proved**.

Timing and Availability

Availability depends on timing, because to continue processing requests, it is necessary to elect and maintain a stable leader.

The required timing condition is:

broadcastTime ≪ electionTimeout ≪ MTBF

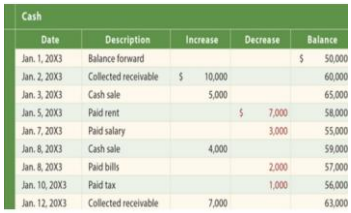
The **broadcastTime**, meaning the time needed to communicate with the other servers, must be much smaller than the **electionTimeout**. In this way, the leader can send heartbeats before the followers start new elections.

The **electionTimeout** must, in turn, be much smaller than the **MTBF**, meaning the mean time between server failures. In this way, when a leader fails, the cluster quickly elects a replacement and remains unavailable only for a short period, rather than for a time longer than the failure itself.

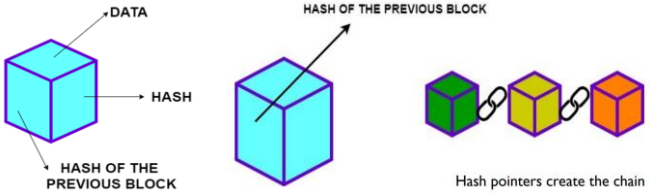
Bitcoin: A Peer-to-Peer Electronic Cash System — Satoshi Nakamoto

Introduction

Ledger: A **ledger** is a register that keeps track of an **ordered sequence of recorded events** and is maintained by a network of **distributed nodes**. It can be **append-only**, meaning that new events can only be added without ever modifying the register.



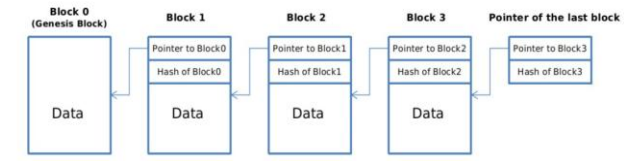
Cash				
Date	Description	Increase	Decrease	Balance
Jan. 1, 20X3	Balance forward			$ 50,000
Jan. 2, 20X3	Collected receivable	$ 10,000		60,000
Jan. 3, 20X3	Cash sale	5,000		65,000
Jan. 5, 20X3	Paid rent		$ 7,000	58,000
Jan. 7, 20X3	Paid salary		3,000	55,000
Jan. 8, 20X3	Cash sale	4,000		59,000
Jan. 8, 20X3	Paid bills		2,000	57,000
Jan. 10, 20X3	Paid tax		1,000	56,000
Jan. 12, 20X3	Collected receivable	7,000		63,000



Blockchain: A **blockchain** is a **distributed** and **immutable ledger** composed of a list of **blocks** connected to each other through **hash pointers**.

Each block in the chain contains the **transactions** performed by the network, together with the block **header**, which contains various **metadata**, a random **nonce** value, and the **hash of the previous block**, which represents a unique reference to that block.

Each node in the network maintains the **same** complete and updated **copy of the ledger**.



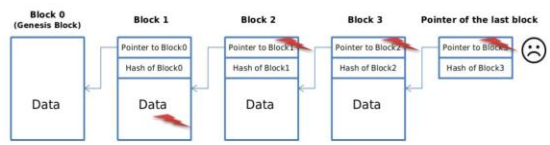

```
Blocco 0:
  Dati:     Alice → Bob (5 BTC)
  PrevHash: 000000000000000    # nessun blocco precedente
  # Hash:   a1b2c3d4            ← calcolato su (Dati ‖ PrevHash)

Blocco 1:
  Dati:     Bob → Charlie (2 BTC)
  PrevHash: a1b2c3d4           # hash del blocco 0
  # Hash:   e5f6g7h8            ← calcolato su (Dati ‖ PrevHash)

Blocco 2:
  Dati:     Charlie → David (1 BTC)
  PrevHash: e5f6g7h8           # hash del blocco 1
  # Hash:   i9j0k1l2            ← calcolato su (Dati ‖ PrevHash)
```

Blockchain Immutability

If an attacker modifies the data in a block **k**, its **hash changes**. Therefore, the **hash pointer** contained in the following block **k+1** no longer matches the previous block.

This creates a **chain effect**: block **k+1** no longer has a valid hash pointer to the previous block. As a consequence, block **k+2** will also have an incorrect hash pointer, and so on.

In the end, the entire **blockchain** will be compromised, and the modification will be **easy to detect**.



To make a malicious modification to a block possible, the attacker must recalculate the **PoW** for every following block, so that all the subsequent **hash pointers** become valid again.

Consensus Problem

All nodes in the network must have the same **consistent copy of the ledger**. Therefore, it is necessary to reach **consensus** on the content of the ledger.

Network Model Assumptions

There is a network of nodes in which anyone can join, read the ledger, and try to append new transactions or blocks to it.

There is no **central authority** that decides whether the ledger is valid, and neither the identity nor the number of participants is known in advance.

Each node can join, leave, or fail at any time. Messages can arrive with any delay, so the network is **asynchronous**.

Some nodes may be **Byzantine**, meaning that, in addition to failing, they may behave maliciously, for example by ignoring the protocol, spreading false information, or trying to create conflicting blocks and transactions.

Consensus

Consensus: The process through which the nodes of the network agree on which **transactions** or **blocks** are valid and in which order they must be added to the shared ledger.

If the majority of the nodes is honest, it will vote against the inclusion of dishonest transactions.

Naive Consensus Through Voting

Each transaction is broadcast to the entire network. The nodes **vote** on the operation, validating whether it is correct or not. If the **majority** of the nodes **approves** it, the operation is considered **valid** and is **added** to the shared ledger.

Sybil Attack

A **Sybil attack** is a technique in which a single attacker **creates multiple false identities**, so that they can vote several times and maliciously **influence** the consensus in their own **favour**.

Double-Spending Attack

Alice creates many false identities in the network, gaining control of more than **50% of the active identities**. Alice tries to spend the same bitcoin twice by sending one transaction to Bob and, at the same time, another transaction to Charlie.

Alice controls the majority of the network, more than **50%**, which allows her to approve the transactions in which she performs **double spending**. The network, relying on the voting mechanism, approves both transactions. For this reason, **consensus** through **voting cannot** be **used**.

Proof of Work Solution

Instead of voting based on the number of participating nodes, consensus in a **PoW blockchain** is determined by the amount of **computational power** that each node provides.

Even if an attacker registers multiple identities, these identities do not add computational power.

A **cryptographic puzzle** that is expensive in terms of computational power is defined. The first node that solves the problem becomes the **leader** for the choice of the block and "wins" the right to decide unilaterally which **valid transactions** to include in the next block of the blockchain and in which order. It also receives a **monetary reward** for its work.

Proof of Work

A miner must find, through **brute force**, a random value **x** called a **nonce**, such that the concatenation of the hash of the created block header and the random nonce is lower than a difficulty threshold **T**: **H(header ‖ x) < T**.

Esempio	Dato	Hash (hex)	Soglia T
1	"block1"	0009077a3b711a272177aad7c46279a616cfbc1b31cf0050f0b37219b43626d9 (SHA-256("block1" ‖ "4779"))	"000"
2	"block2"	0005e6004615e7409b9c88d089dd09f4542f98976cc40f8d918a22b188c38cd9 (SHA-256("block2" ‖ "2694"))	"000"
3	"block3"	0005e531415c92c39d2421f1ec4e5310d0b5c36e06c692bc29bceed0cf7db8f4 (SHA-256("block3" ‖ "1"))	"000"

The search is based on random attempts, so it requires a high number of operations. Once the nonce value has been found, anyone can quickly verify it by calculating the hash and comparing it with the target threshold.

MemPool

Each node keeps a temporary memory in **RAM** called the **MemPool**, where all the Bitcoin transactions received from the network are collected while waiting to be confirmed and added to the ledger.

If a node receives two conflicting transactions, it removes from its MemPool the one that is identified as **double spending**.

Mining

Mining means **competing** to add the **transactions** from its own **MemPool** to the next block of the ledger.

The miner collects the valid transactions from its own MemPool and inserts them into a **candidate block**, selecting valid transactions with the highest fees.

Once the transactions have been selected, the node creates the **block header** and starts the **PoW process**. The winner of the PoW adds its candidate block to the blockchain and broadcasts the updated version of the ledger to nearby nodes, including the new block.

When a node receives the block, it calculates the hash of the received **block header** and checks that this hash is lower than the established **difficulty target**. It also checks that the transactions inside the block are valid. If they are not valid, the block is not accepted.

If the block passes verification, each node **adds** it to its **own copy** of the blockchain, updating the transaction ledger. The node removes from its MemPool all the transactions included in

the block. In addition, if its MemPool contains transactions that conflict with those in the block, they are deleted.

Attack Risks

To alter the blockchain by modifying a block, it would be necessary to redo the **PoW** of all the blocks following the one that the attacker wants to modify.

The **probability** of **solving** the cryptographic puzzle is **proportional** to the amount of **computational power** provided. To alter the blockchain in its own favour with high probability, an attacker must control at least **51% of the total computational power** available.

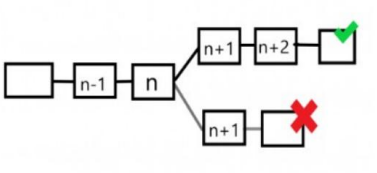
In addition, for every block mined through PoW, miners receive a **coinbase reward**. Bitcoin is designed to make it **less convenient** for a miner to alter the blockchain by **investing** an enormous amount of **computational power** than to continue **receiving** monetary rewards.

Assumption: It is assumed that the nodes controlling the majority of the **computational power** are honest.

Nakamoto Consensus

If two nodes solve the **PoW** almost at the same time, each of them creates a block attached to the same previous block, creating two **parallel forks** in the blockchain.

Later, as new blocks are added, the network determines which branch of the blockchain will become the main one, considering valid the branch that accumulates the **greatest** amount of **computational work**, that is, the **longer chain** of the two.



In fact, miners consider valid the fork to which they append their new block. Honest miners choose and append their blocks to the chain that **accumulates** the **greatest** amount of computational **work**.

After a fork, there will eventually be a moment when only one of the two branches is extended by the miners, becoming the longer chain and therefore resolving the fork.

The transactions contained in the block of the abandoned branch that are not present in the winning branch are added back to the **MemPool** of each node.

“6 Confirmations” Rule

A transaction is considered **probabilistically secure** only when at least **5 blocks** have been **added** after the block containing that transaction.

This rule does not mean that the block is **mathematically final**, because it could still be **removed** if an alternative chain managed to accumulate **more PoW work**.

However, the **probability** of a reorganization **decreases** as new blocks are added. All honest miners support the **longest chain**. A dishonest miner who wants to propose an alternative chain has **more computational** work to **perform** after every **new block added** to the honest chain, in order to **create** a **longer chain**.